

Mininet Tutorial

Build, test, and debug virtual SDN networks on one machine

Hands-on lab deck

OpenFlow + OVS

Controller-ready

```
$ sudo mn --topo single,3 --mac  
--switch ovsk --controller  
remote
```

```
mininet> nodes  
mininet> net  
mininet> pingall  
mininet> h1 ip addr  
mininet> exit
```

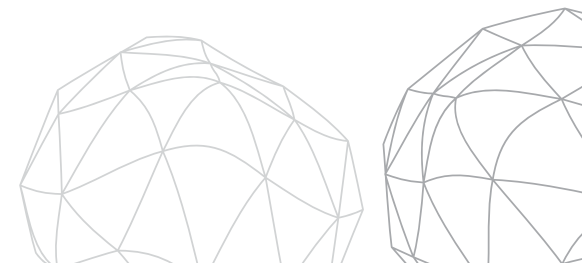
What students will be able to do

create topologies • inspect links • run traffic • write a client-server program

Hands-on using mininet

Presented by Bishal Kumar Nath

Research Assistant,
Meity sponsored project ISEA Phase III
Department of Computer Science & Engineering
Tezpur University



Session Overview



Expected Outcome:

Participants gain practical insight into Experimentation using Mininet

Part 1 : Introduction mininet

Part 2 : Installing mininet

Part 3 : mininet Commands

Part 4 : Example: Client-Server Program

Part 5 : Flow tables

Simulation/ Emulation tools for Network Research and development

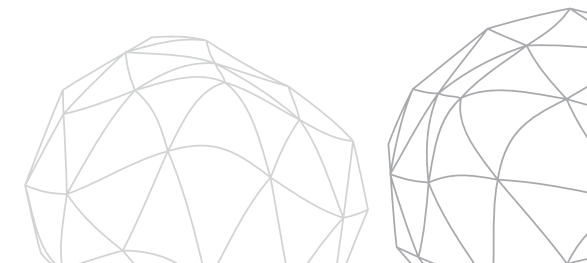
Traditional network testing tools

- 1) ns-3 :
 - a **simulation tool**
 - Limited capability
 - No real time monitoring
- 2) Mininet:
 - Emulation tool
 - Constraint by system resources
- 3) P4 development tool:
 - Current industry attraction.
 - Mainly data plane programming tool
 - Real developing tool, not ideal for testing

Aspect	Simulation (ns-3)	Emulation (Mininet)
Definition	Models network behavior mathematically and programmatically	Runs actual code/protocols in a controlled virtual environment
Realism	Abstracted; approximates network behavior	High realism; uses real kernel, TCP/IP stack, and applications
Performance	Faster, scalable to large topologies	Slower, limited by host machine resources
Use Case	Research, theoretical analysis, protocol design/testing	System testing, prototyping, SDN controller evaluation
Flexibility	Easy to modify parameters, inject custom models	Limited to what the underlying OS and virtualization support
Examples	ns-3, OMNeT++	Mininet, GNS3

Part 1:

Introduction to mininet



Introduction to mininet

What is mininet?

Mininet is a lightweight **network emulator** that creates realistic virtual networks on a single machine.

Mininet is widely used for **SDN teaching, research, and prototyping**.

Why Mininet?

- **Low-cost experimentation** — no physical testbed required
- **Rapid topology creation** — build networks in seconds
- **Supports SDN experiments** — works with OpenFlow and SDN controllers
- **Repeatable experiments** — easy to modify and rerun
- **Realistic behavior** — useful for testing controller logic and flow rules

<https://mininet.org/>

a practical virtual lab for building and testing SDN networks quickly and safely



What to deal with?

A network emulator for rapid SDN experimentation



Hosts

Linux network namespaces with separate IP, interfaces, and processes.



Switches

Usually Open vSwitch instances controlled by OpenFlow rules.



Links

Virtual Ethernet pairs with configurable bandwidth, delay, and loss.

Overview of Mininet Architecture

Mininet creates a **virtual SDN network** on a single machine by emulating the main components of a real network.

Core Components

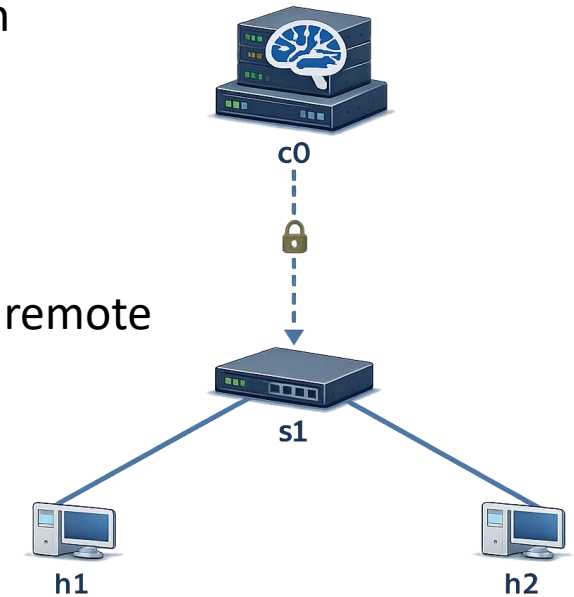
- **Hosts** — end systems that generate and receive traffic
- **Switches** — forward packets, typically using **Open vSwitch**
- **Controllers** — provide SDN control logic. A Mininet switch can connect to a remote controller:
 - `sudo mn --controller=remote,ip=127.0.0.1,port=6653`
- **Links** — virtual connections between network nodes

How Mininet Works

- Uses **Linux network namespaces** to isolate hosts
- Uses **virtual Ethernet (veth) pairs** to create links
- Supports **OpenFlow-based switches** for SDN experiments

Key Idea

Mininet provides a realistic and flexible environment for building and testing SDN topologies without physical hardware.



Custom topology in Python

Use Python when built-in topologies are not enough

```
from mininet.topo import Topo

class MyTopo(Topo):
    def build(self):
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        s1 = self.addSwitch('s1')
        self.addLink(h1, s1, bw=10, delay='5ms')
        self.addLink(h2, s1, bw=10, delay='5ms')

topos = {'mytopo': MyTopo}
```

Run it

```
sudo mn --custom mytopo.py --topo
mytopo --link tc
```

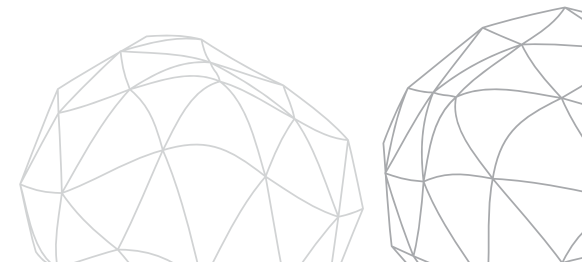
--link tc activates bandwidth, delay, loss, and queue parameters.

Lab idea: change bw=10 to bw=1 and observe iperf.

Part 2:

Installing mininet

Meity ISEA Phase 3



Installation options

Choose the setup that matches your classroom

Official VM

Best for beginners

Prebuilt environment; fewer dependency problems.

Native Linux

Best for researchers

Install on Ubuntu/Debian; easiest to integrate with tools.

Cloud VM

Best for remote labs

Give students SSH access to identical machines.

```
# Native Ubuntu example
sudo apt update
sudo apt install mininet openvswitch-switch
sudo mn --test pingall
sudo mn -c # cleanup after experiments
```

Installing mininet

➤ Common Method:

- ❑ `sudo apt update`
- ❑ `sudo apt install mininet`

Running Mininet

➤ Start the default topology:

- ❑ `sudo mn`
- This usually creates:
 - **2 hosts**
 - **1 switch**
 - **1 controller**

➤ Useful Startup Options

- ❑ `sudo mn --topo single,3`

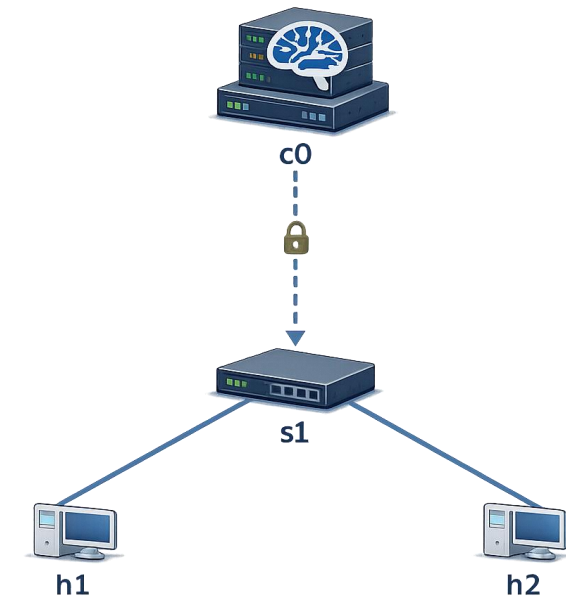
➤ Creates:

➤ **1 switch**

➤ **3 hosts**

- ❑ `sudo mn --controller remote`

➤ Connects Mininet to a **remote SDN controller**



Key Idea:

Mininet can create a working SDN test network in seconds using simple commands.

Creating Simple SDN Network Topologies

Mininet allows us to create different **network topologies** quickly using simple command-line options.

Common Topology Types

❖ Single Switch Topology

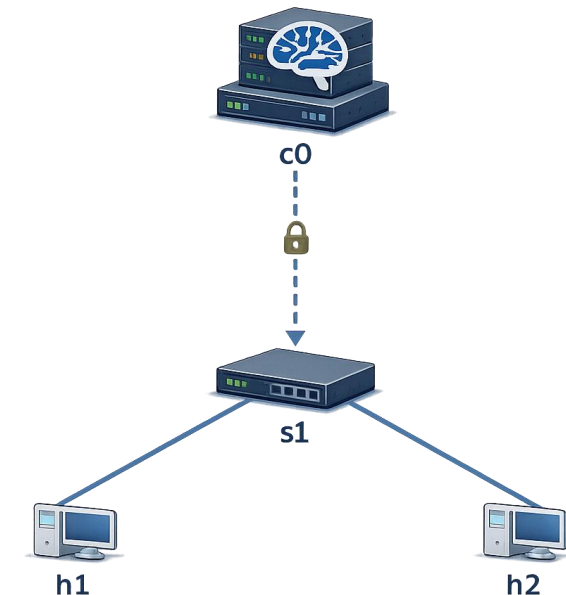
- One switch connected to multiple hosts
- Eg: Single switch with 3 hosts
 - ❑ `sudo mn --topo single,3`

❖ Linear Topology

- Multiple switches connected in a chain
- Eg: Linear topology with 3 switches
 - ❑ `sudo mn --topo linear,3`

❖ Tree Topology

- Hierarchical structure with multiple levels of switches
- Eg: Tree topology
 - ❑ `sudo mn --topo tree,depth=2,fanout=2`



Custom Topology Creation in Mininet

Why Custom Topologies?

- Model **realistic or research-specific network scenarios**
- Design the exact arrangement of **hosts, switches, and links**
- Study controller behavior in **user-defined SDN environments**
- Go beyond default **single, linear, and tree** topologies

How It is Done

- Custom topologies are typically written in **Python**
- The user defines:
 - **Hosts**
 - **Switches**
 - **Links**
- The topology can then be launched in Mininet using a custom script

Example Workflow

1. Write a Python topology file
2. Define network nodes and links
3. Launch the topology using Mininet

Key Idea

Custom topology creation makes Mininet flexible for SDN research, teaching, and rapid prototyping.



Example: Python Custom Topology

A custom topology in Mininet can be created by defining a Python class that inherits from Topo.

Example Code:

```
from mininet.topo import Topo
```

```
class MyTopo(Topo):  
    def build(self):  
        h1 = self.addHost('h1')  
        h2 = self.addHost('h2')  
        s1 = self.addSwitch('s1')
```

```
        self.addLink(h1, s1)  
        self.addLink(h2, s1)
```

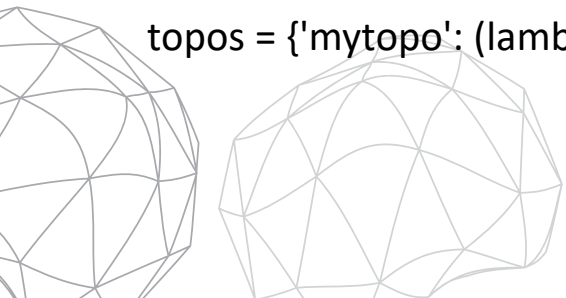
```
topos = {'mytopo': (lambda: MyTopo())}
```

What This Topology Creates

- **2 hosts** → h1, h2
- **1 switch** → s1
- Links between:
 - h1 and s1
 - h2 and s1

Running the Topology

```
❏ sudo mn --custom mytopo.py --topo  
mytopo
```



Running and Verifying a Custom Topology

A custom topology in Mininet can be created by defining a Python class that inherits from Topo.

Verifying the Topology

- **Check nodes**
 ❑ `nodes`
- **View network links**
 ❑ `net`
- **Test connectivity**
 ❑ `pingall`
- **Test between two hosts**
 ❑ `h1 ping h2`

Why This Step Matters

- Confirms the topology was created correctly
- Verifies communication between hosts
- Helps debug topology design and connectivity issues



Inspecting Flow Rules in Mininet

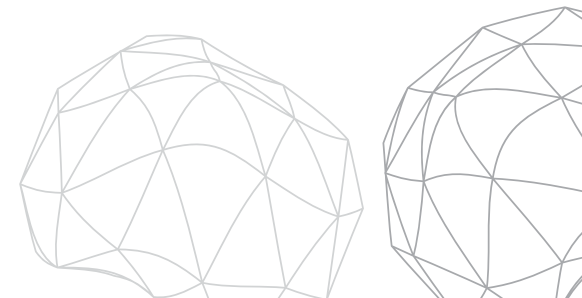
In Mininet, switches are most commonly implemented using **Open vSwitch (OVS)**, which provides the **programmable data plane** required for SDN experiments.

Verifying the Topology

- **Check nodes**
□ nodes
- **View network links**
□ net
- **Test connectivity**
□ pingall
- **Test between two hosts**
□ h1 ping h2

Why This Step Matters

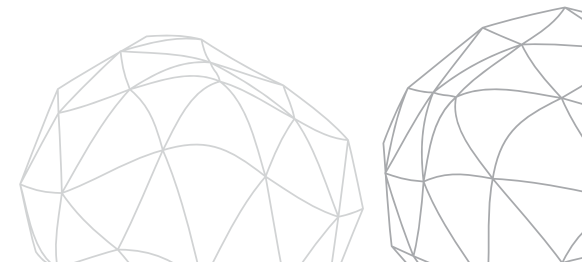
- Confirms the topology was created correctly
- Verifies communication between hosts
- Helps debug topology design and connectivity issues



Part 3:

Mininet commands

Meity ISEA Phase 3



Mininet commands

Running Mininet

- Start the default topology:

❏ `sudo mn`

- This usually creates:

- **2 hosts**
- **1 switch**
- **1 controller**

Specify controller type

`sudo mn --controller=remote`

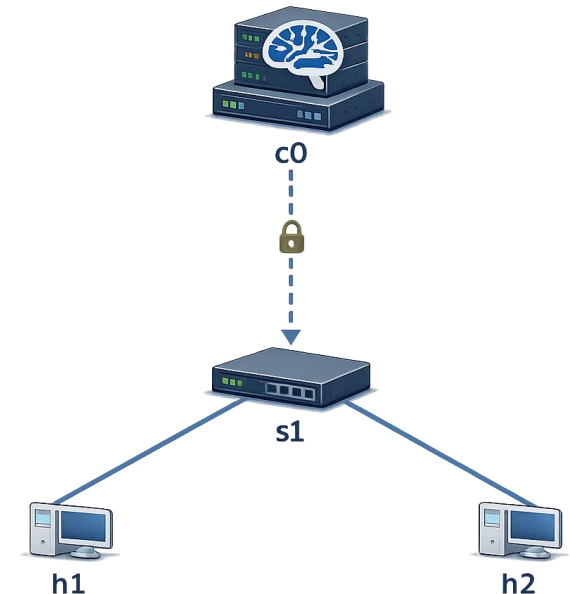
Tree topology

`sudo mn --topo`

`tree,depth=2,fanout=2`

➤ Cleanup Commands

- Remove old Mininet configurations
- `sudo mn -c`



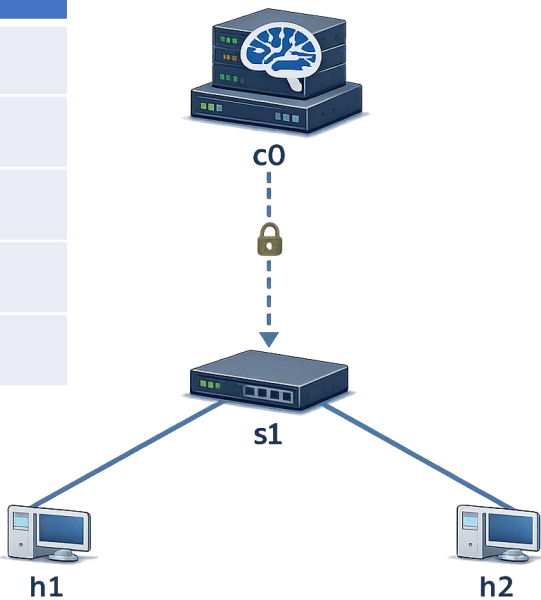
Key Idea:

Mininet can create a working SDN test network in seconds using simple commands.

Mininet commands

1. Starting Mininet

Command	Description
<code>sudo mn</code>	Start the default topology (2 hosts, 1 switch, 1 controller)
<code>sudo mn --topo single,4</code>	Create a single-switch topology with 4 hosts
<code>sudo mn --topo linear,3</code>	Create a linear topology with 3 switches
<code>sudo mn --topo tree,depth=2,fanout=2</code>	Create a tree topology
<code>sudo mn --controller remote</code>	Connect to a remote SDN controller



Mininet commands

2. Network Information Commands

Command	Description
nodes	Display all nodes in the network
net	Show network connections
dump	Display detailed information about all nodes
links	Show all network links
hosts	List all hosts



Mininet commands

3. Connectivity Testing

Command	Description
pingall	Test connectivity between all hosts
h1 ping h2	Ping from host h1 to host h2
h1 ping -c 5 h2	Send 5 ping packets from h1 to h2



Mininet commands

3. Connectivity Testing

Command	Description
pingall	Test connectivity between all hosts
h1 ping h2	Ping from host h1 to host h2
h1 ping -c 5 h2	Send 5 ping packets from h1 to h2



Mininet commands

4. Host Management Commands

Command	Description
xterm h1	Open a terminal window for host h1
xterm h1 h2	Open terminal windows for multiple hosts
h1 ifconfig	Display network interface information
h1 ip addr	Show IP address configuration
h1 route -n	Display routing table



Mininet commands

5. Performance Evaluation Commands

Command	Description
iperf	Measure TCP throughput between hosts
iperf h1 h2	Run throughput test between h1 and h2
iperfudp	Measure UDP performance



Mininet commands

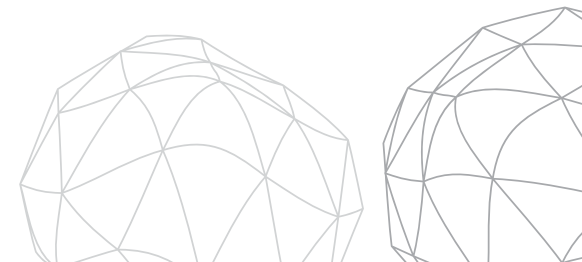
6. Cleanup and Exit Commands

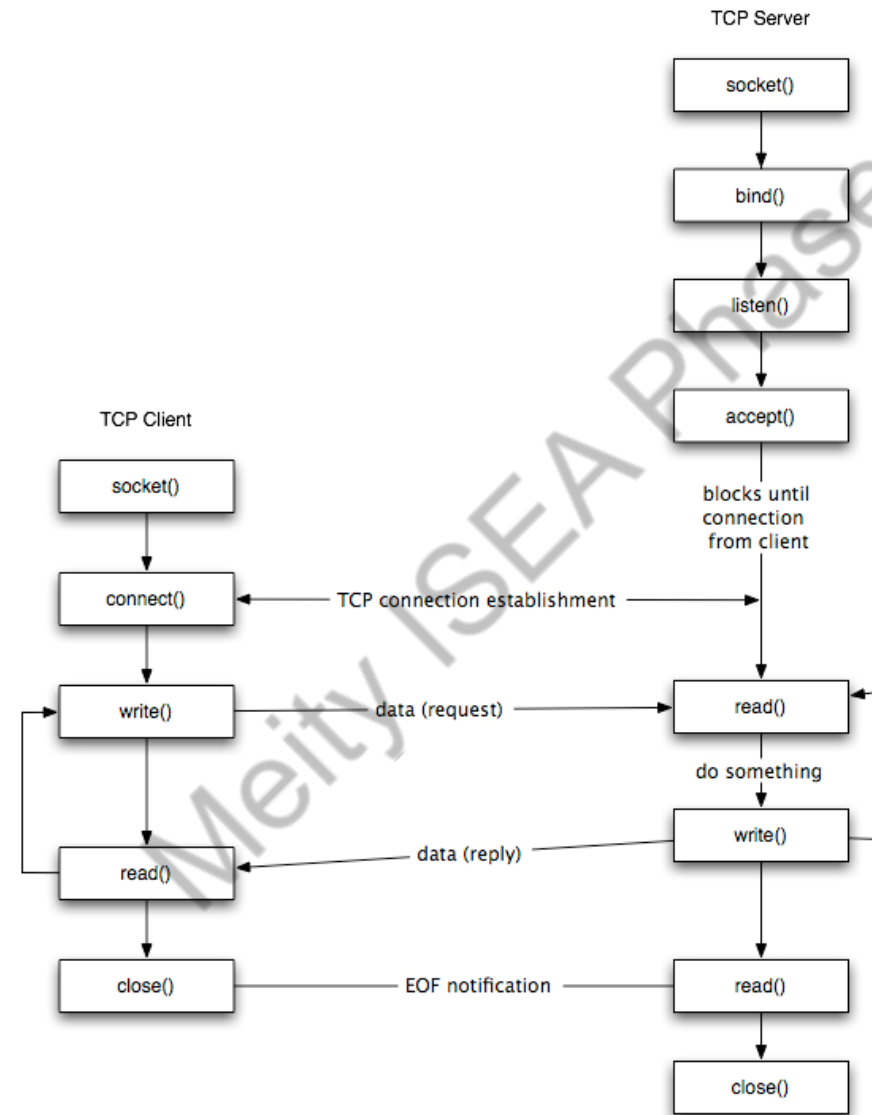
Command	Description
exit	Exit Mininet CLI
quit	Exit Mininet CLI
sudo mn -c	Clean up Mininet processes and configurations



Part 4:

Server – client: An example program





Running a Simple Client–Server Application

Scenario:

h1 = Server

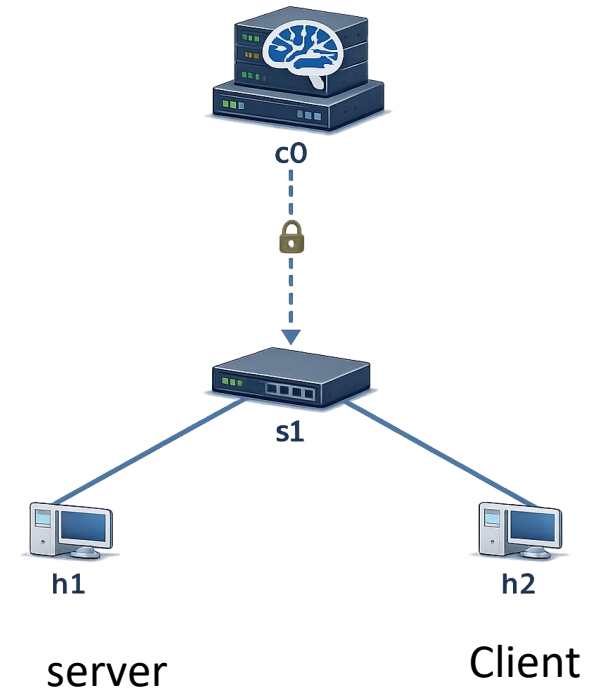
h2 = Client

Topology = Mininet default topology

Command = sudo mn

Default topology:

h1 ---- s1 ---- h2



Running a Simple Client–Server Application

Scenario:

h1 = Server

h2 = Client

Topology = Mininet default topology

Command = sudo mn

Default topology:

h1 ---- s1 ---- h2

Command:

 nodes

available nodes are:

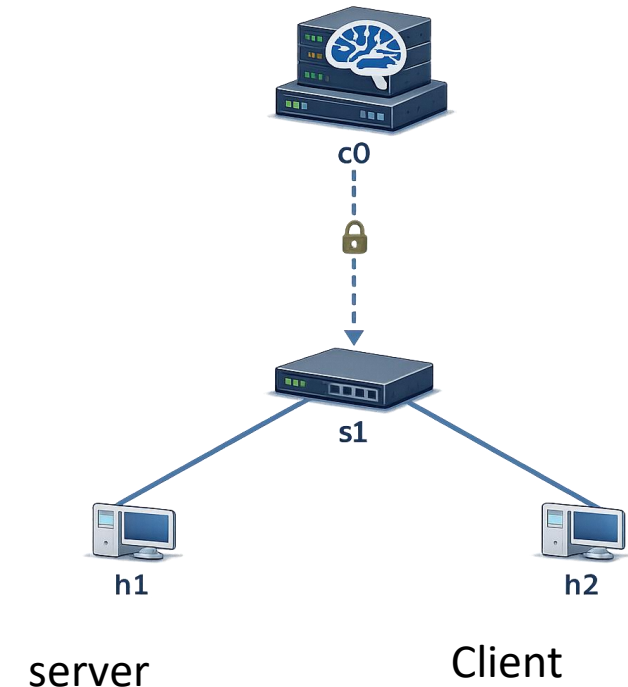
c0 h1 h2 s1

Where:

h1 and h2 are virtual hosts

s1 is a virtual switch

c0 is the default controller



Running a Simple Client–Server Application

Command:

☐ Net

h1 h1-eth0:s1-eth1

h2 h2-eth0:s1-eth2

s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0

Command:

☐ h1 ifconfig

☐ h2 ifconfig

h1 = 10.0.0.1

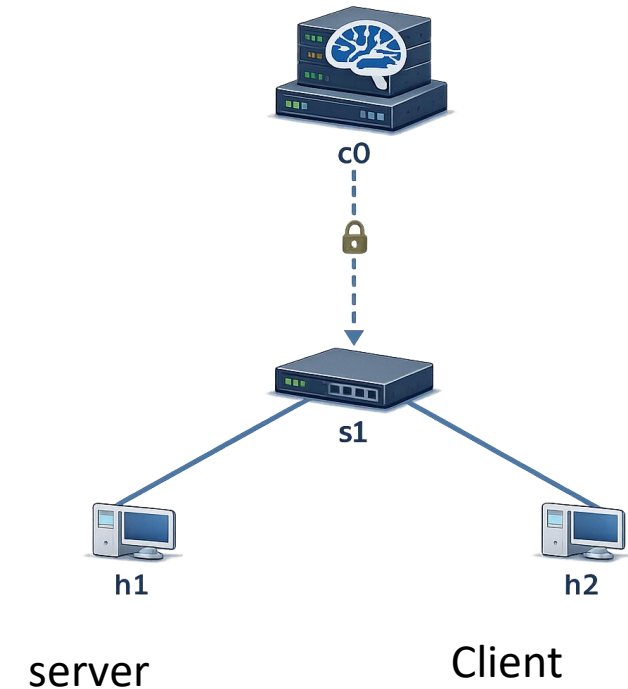
h2 = 10.0.0.2

Command:

☐ h1 ping h2

☐ pingall

*** Results: 0% dropped



Python Server Code for h1

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind(("10.0.0.1", 5000))
server.listen(1)

print("Server is running on h1...")
print("Waiting for client connection...")

conn, addr = server.accept()
print("Connected by:", addr)

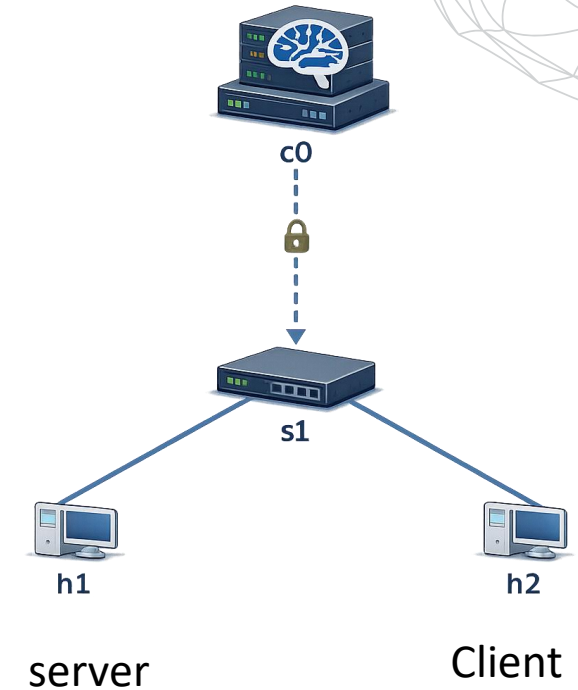
data = conn.recv(1024).decode()
print("Client says:", data)

conn.send("Hello h2, message received by h1 server".encode())

conn.close()
server.close()
```

The logic:

1. Create socket
2. Bind socket to IP and port
3. Listen for client
4. Accept connection
5. Receive message
6. Send response
7. Close connection



Python Client Code for h2

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect(("10.0.0.1", 5000))

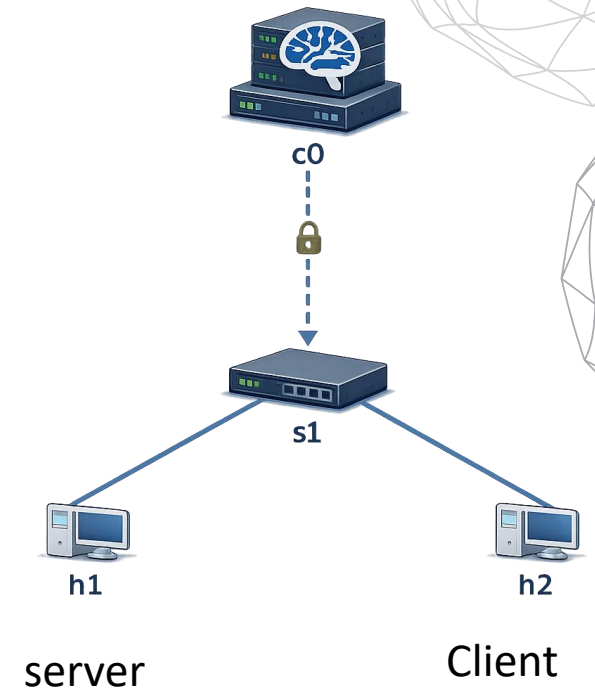
client.send("Hello h1, this is h2 client".encode())

response = client.recv(1024).decode()
print("Server says:", response)

client.close()
```

The logic:

1. Create socket
2. Connect to server IP and port
3. Send message
4. Receive response
5. Close connection



How to Run

use:

```
❏ xterm h1 h2
```

This opens separate terminals for h1 and h2.

Server-side

In the h1 terminal:

```
❏ python3 server.py
```

Expected output on h1:

```
Server is running on h1...  
Waiting for client connection...  
Connected by: ('10.0.0.2', some_port)  
Client says: Hello h1, this is h2 client
```

Client-side

In the h2 terminal:

```
❏ python3 client.py
```

Expected output on h2:

```
Server says: Hello h2, message  
received by h1 server
```

Common mistakes and quick fixes

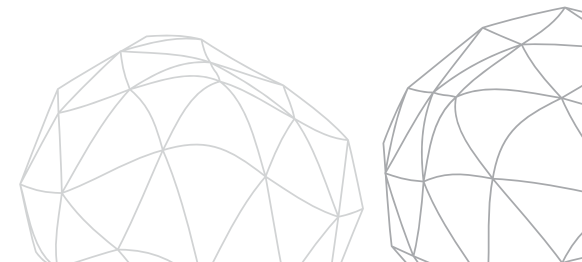
Keep this as your last-slide reference during live demos

Mistake	Symptom	Fix
Old topology still running	Cannot create interfaces	<code>sudo mn -c</code>
Controller not connected	pingall fails	Check IP, port, protocol
Wrong OpenFlow version	No flows installed	Use <code>protocols=OpenFlow13</code>
No sudo	Permission error	Run Mininet with sudo
VM has no internet	Install fails	Check NAT/bridge adapter

Part 3:

Flow tables

Meity ISEA Phase 3



Installing and Monitoring Flow Rules

Once an OVS bridge is created, we can **install flow rules** and **monitor switch behavior** using OpenFlow tools.

Installing a Flow Rule

- **Example: normal switching**

```
❏ sudo ovs-ofctl add-flow br0 actions=normal
```

- **Example: forward packets to a port**

```
❏ sudo ovs-ofctl add-flow br0 in_port=1,actions=output:2
```

Monitoring Flow Rules

- **View installed flow entries**

```
❏ sudo ovs-ofctl dump-flows br0
```

- This shows:

- Match fields
- Actions
- Packet and byte counters
- Priority information

Flow Table

Flow ID	Match Fields	Action	Priority	Counters
1	Src IP = 10.0.0.1, Dst IP = 10.0.0.5	Forward → Port 3	100	Packet Count
2	TCP Dst Port = 80	Forward → Port 2	90	Packet Count
3	Src IP = 10.0.0.3	Drop Packet	80	Packet Count
4	Any	Send to Controller	0	Packet Count

Simple Example

Example flow rule:

➤ Match:

- Source IP = 10.0.0.1
- Destination IP = 10.0.0.5

➤ Action:

- Forward packet to **Port 3**

➤ Priority

- Determines which rule is applied when multiple matches occur

➤ Counters

- Maintain statistics such as number of packets and bytes processed

Flow Table in Abstract

Flow 1.	Rule (exact & wildcard)	Action	Statistics
Flow 2.	Rule (exact & wildcard)	Action	Statistics
Flow 3.	Rule (exact & wildcard)	Action	Statistics
.....			
Flow N.	Rule (exact & wildcard)	Default Action	Statistics

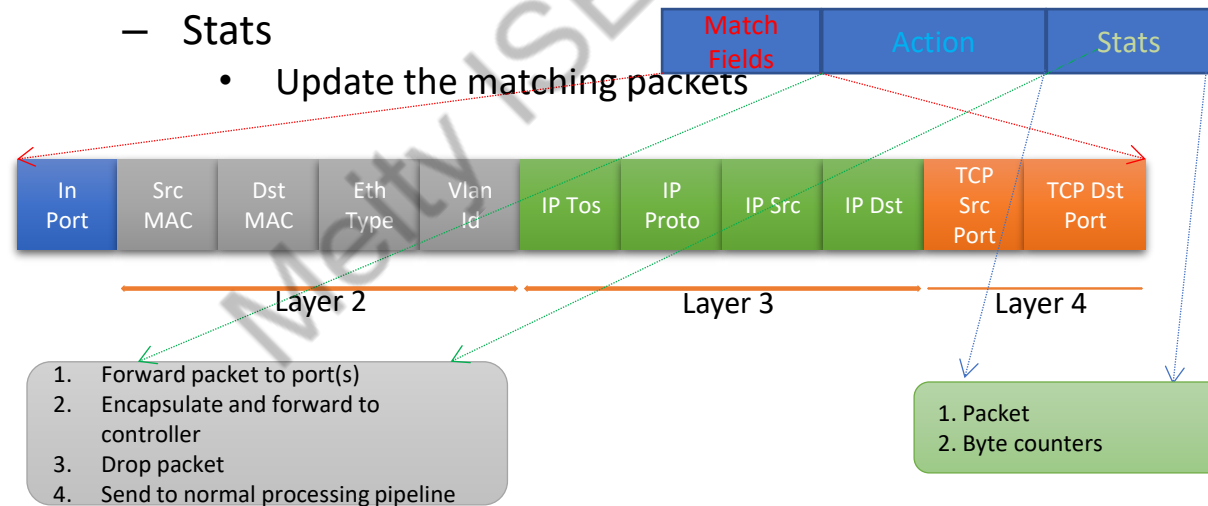
Simple Example

Example flow rule:

- Match:
 - Source IP = 10.0.0.1
 - Destination IP = 10.0.0.5
- Action:
 - Forward packet to **Port 3**

Flow Entry

- A flow entry consists of
 - Match fields
 - Match against packets
 - Action
 - Modify the action set or pipeline processing
 - Stats
 - Update the matching packets



Example

Switching

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	00:1f:..	*	*	*	*	*	*	*	port6

Flow Switching

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
port3	00:20:..	00:1f:..	0800	vlan1	1.2.3.4	5.6.7.8	4	17264	80	port6

Firewall

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	*	*	*	*	*	*	*	22	drop

Routing

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	*	*	*	*	5.6.7.8	*	*	*	port6

VLAN Switching

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	00:1f:..	*	vlan1	*	*	*	*	*	port6, port7, port9

Hands-on

Meity ISEA Phase 3



Thank You For Joining Us

We hope this lecture inspired curiosity, equipped you with knowledge, and encouraged you to explore further digital adventures excitingly, together.

